Introduction to Embedded Systems

Course Code: 244066

Chapter 1: Overview of Embedded Systems

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

1. Overview
2. Introduction
3. Embedded Products
4. Characteristics
5. Significance
6. Advantages
7. Architecture
8. Components
9. Data Flow
10. Building Blocks
11. Embedded Processors
12. Embedded Operating Systems
13. Development Tools & Methods
14. Programming Languages
15. Classification
16. Summary

1.1. Overview

Prerequisites: Advanced Programming, Assembly Language, Microcomputer Principles and Interface Technology

Textbooks & References

- **Primary**: *Qian Xiaojie, Cheng Nan, Introduction to Embedded Systems* (2017).
- **Supplementary**: *Ning Yang, Zhou Yulin, Fundamentals and Applications of Embedded Systems* (2012).
- **Reference**: *Liu Huoliang, Yang Sen, Practical Guide to STM32 Library Development* (2014).

1.1. Overview

Course Objectives

- Understand embedded processor architecture, memory mapping, and peripheral hardware.
- Gain hands-on skills in embedded development platforms, drivers, and interfaces.

Goal is to understand, design, and program embedded systems.

1.1. Overview

Course Content (8 Chapters)

1. Overview of Embedded Systems
2. STM32 Microcontrollers
3. General I/O Ports of STM32
4. CM3 Exceptions and STM32 Interrupts
5. STM32 Serial Communication Interface
6. STM32 Direct Memory Access (DMA)
7. STM32 Timing Interface
8. STM32 Analog Interface

1.1. Overview

Assessment & Evaluation

- **60%**: Comprehensive course project (Code , Report, Presentation).
- **40%**: Class participation (Attendance, Quiz, Assignments).

Comprehensive Course Project:

- Independent hardware/software design.
- Report including project background, system framework, hardware/software design, results, and presentation.

1.1. Overview

- If you have any questions regarding this course, use the class discussion group.
- For class related issues, please send me an email with [ES24] in subject line.

Email: Imran.Nadeem@outlook.com

1.1. Overview

Academic Honesty

✓ Do’s

- Start assignments early to manage time effectively
- Can use books, reference materials and other online resources for guidance and brainstorming
- Develop your own understanding and apply techniques independently

✕ Don’ts

- Don’t copy directly from GPT, LLMs, or any other source
- Don’t leave assignments until the last minute
- Don’t ignore originality and assignment requirements

1.1. Overview

Late submission policy

- You should hand your assignments in on time.
- If you hand in your work late, your mark will be capped based on how many days late it is:
  - 1 day late- mark capped at 75%
  - 2 day late- mark capped at 50%
  - 3 day late- mark capped at 25%
  - More than 3 days late- no marks available

“Push yourself harder than yesterday,
if you want a different tomorrow.”

What is a System?

1.2. Introduction

- A **system** is a structured way of doing things
- It organizes one or more tasks according to a plan, program, or set of rules.
- It can be thought of as a collection of **interconnected parts** that work together to achieve a specific goal.

1.2. Introduction

Examples:

- A **university system** (students, teachers, courses, exams working together for education).
- A **transport system** (buses, trains, traffic lights all coordinating for safe travel).
- A **computer system** (hardware + software working together to process information).

What is an Embedded System?

1.2. Introduction

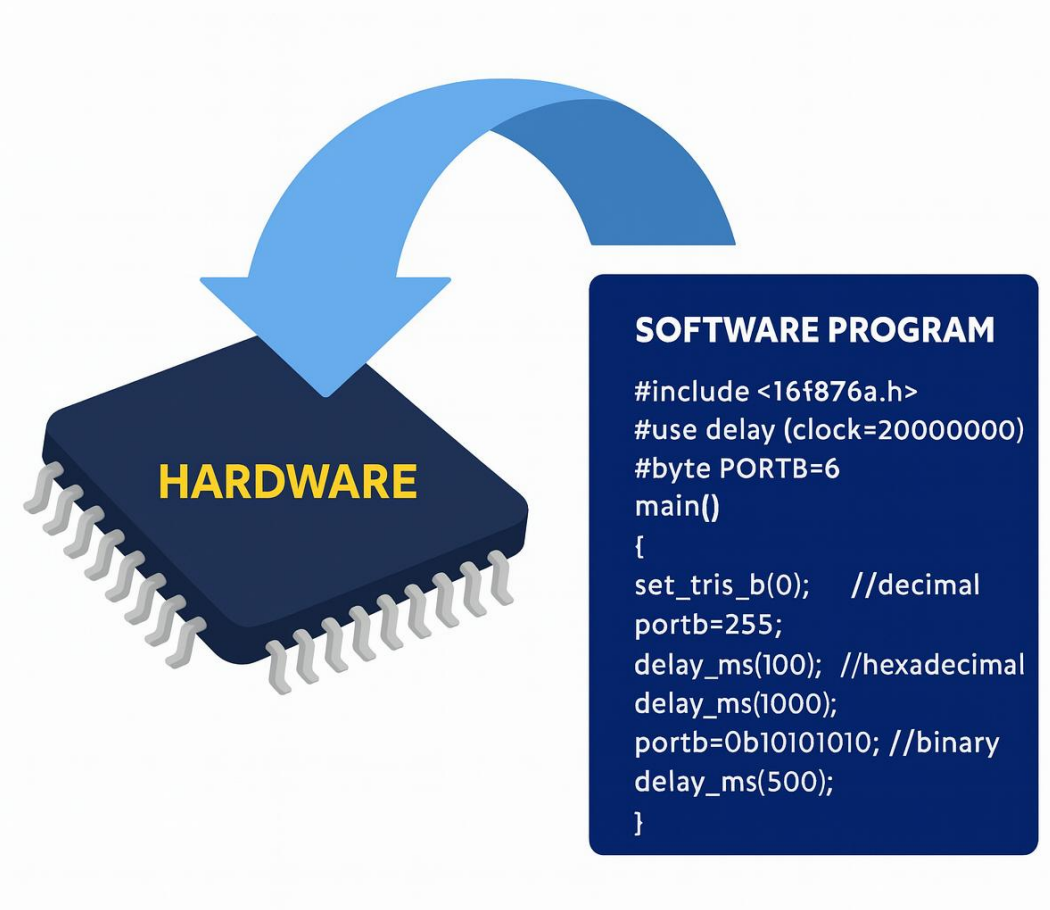
An **embedded system** is a **computer-based system** that has both hardware and software working together as one unit.

1.2. Introduction

Unlike a general-purpose computer (like a PC), an embedded system is **built for a specific task**.

It consists of:

- **Hardware** → Microcontrollers, sensors, actuators, circuits.
- **Software** → Special programs (firmware) that control the hardware.



1.3. Embedded Products



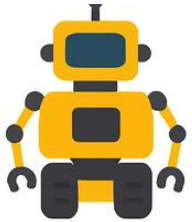
Washing machines (control software + motor hardware).



Cars (engine control unit, airbags, ABS, self driving).



Smartphones (camera control, battery management).

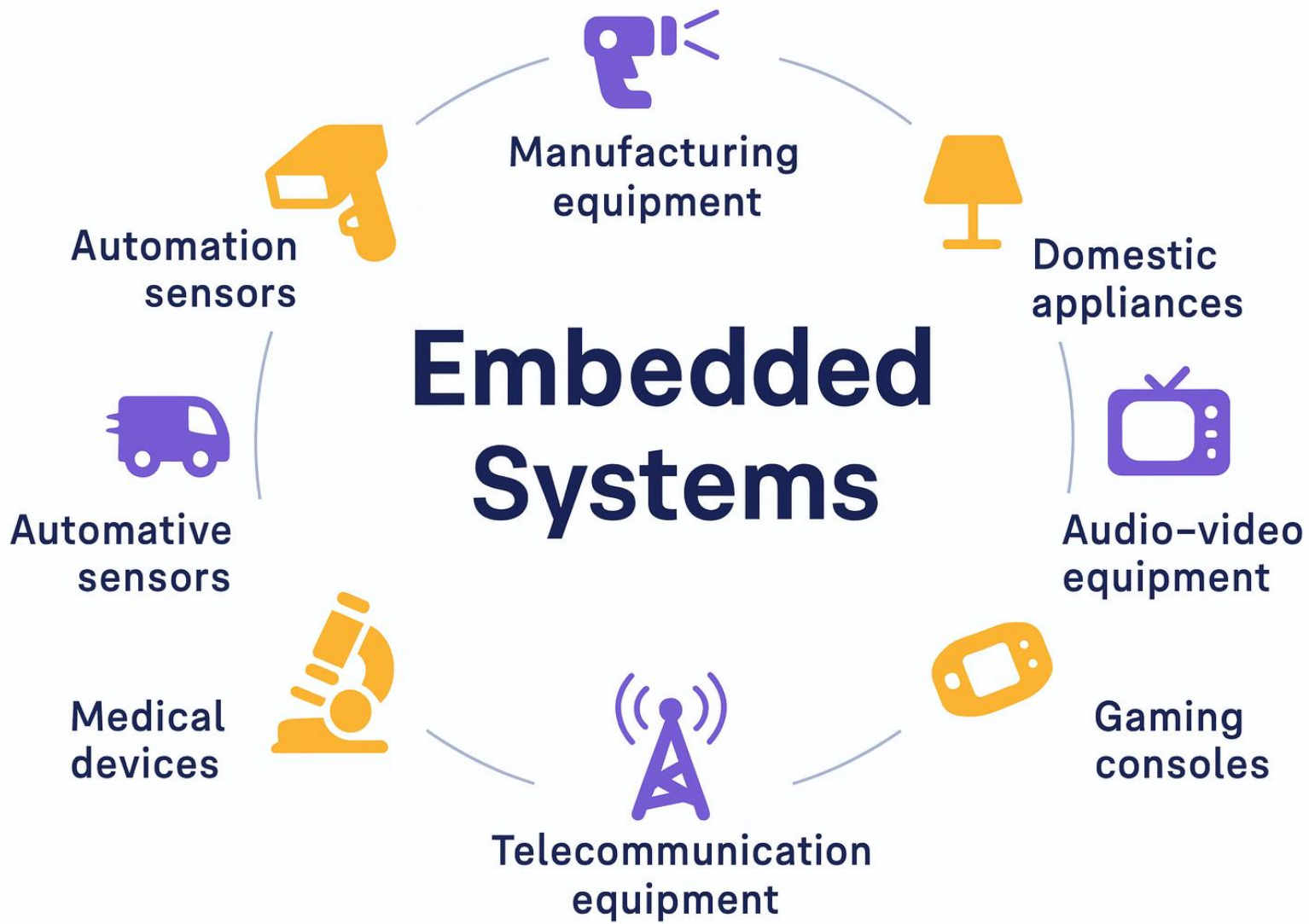


Robotics (control sensors, actuators, motors, and communication)



Drones (UAVs) (flight control, GPS navigation, obstacle detection, camera stabilization, and communication

1.3. Embedded Products



1.4. Characteristics

- **Dedicated functionality** (single-task / limited tasks).
- **Real-time operation** (must meet timing constraints).
- **Resource-constrained** (limited CPU, memory, power).
- **Reliability & stability** (continuous operation).
- **Low power consumption** (especially in portable devices).
- **Tight hardware-software integration**.

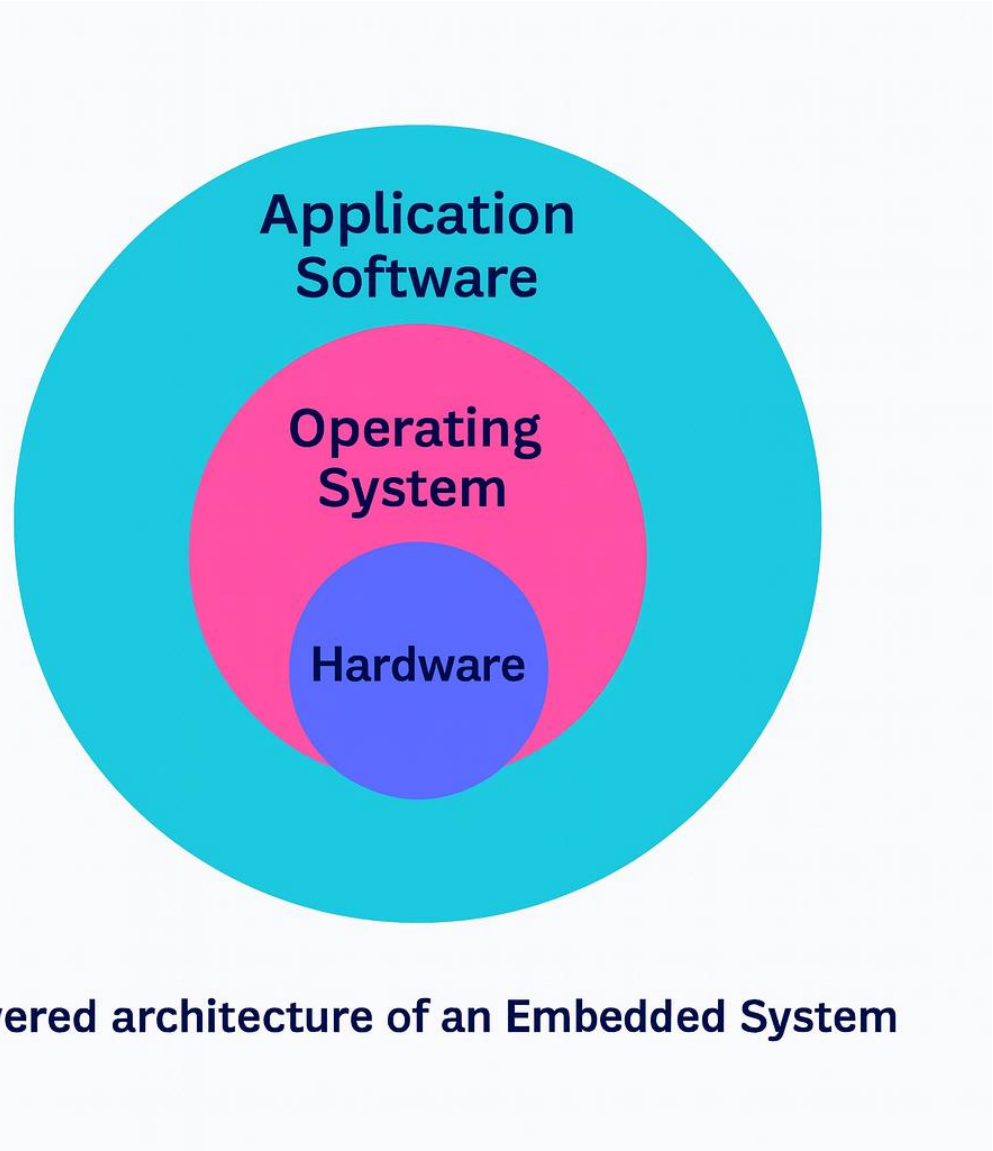
1.5. Significance

- Embedded systems have become **indispensable** in modern life.
- Their **compact size**, **low cost**, and **efficient design** make them the preferred choice in almost every field.
- Embedded systems improve **efficiency, safety, and convenience**.

1.6. Advantages

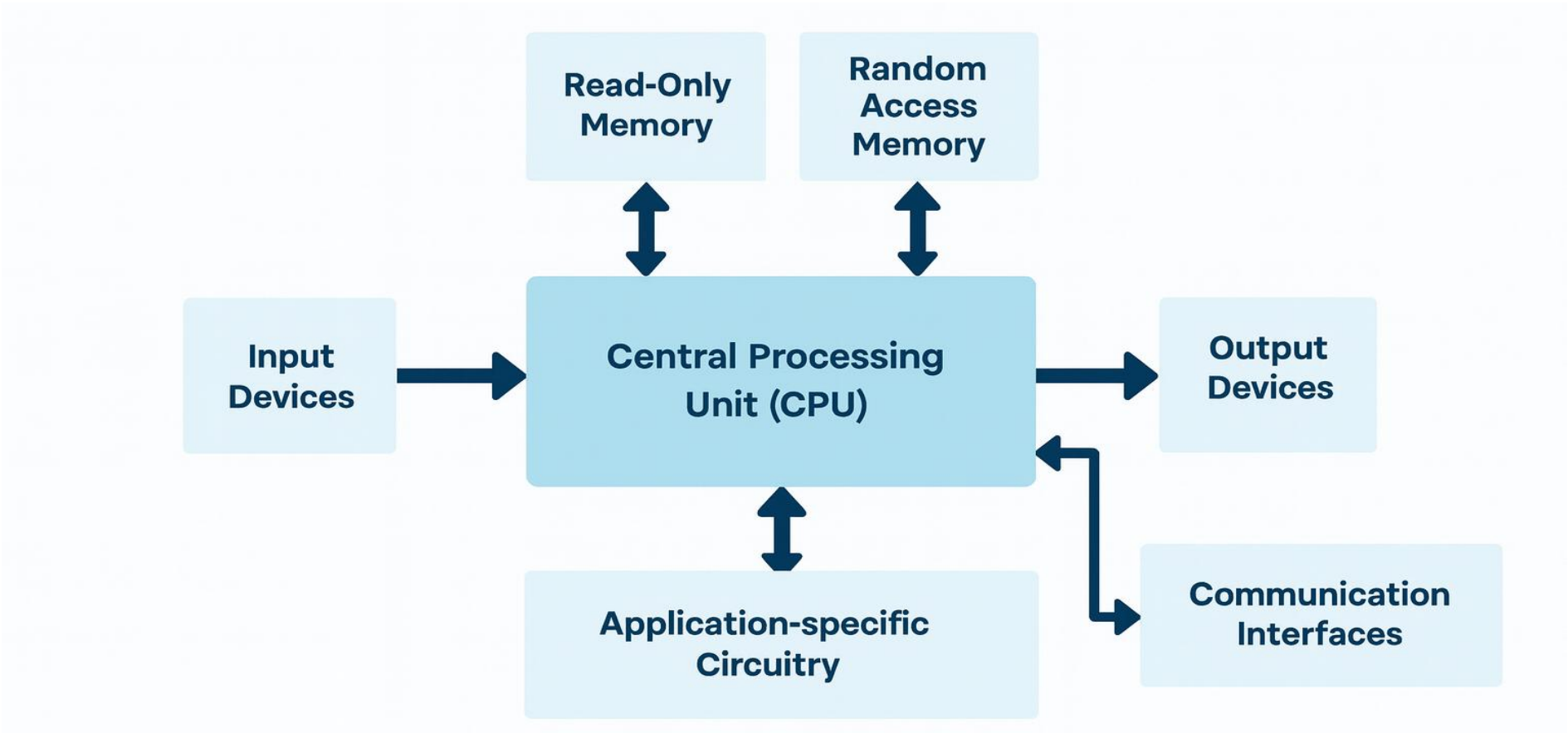
- **Affordability** → Cost-effective solutions make advanced technology available to everyone.
- **Miniaturization** → Enables compact devices like smartwatches, IoT sensors, and portable medical tools.
- **Dependability** → Ensures **continuous operation** in systems where failure is not an option (e.g., pacemakers, avionics).
- **Driving Innovation** → They are the **backbone of IoT, AI devices, and smart technology**, shaping the future of automation and digital life.

1.7. Architecture



Layered architecture of an Embedded System

1.9. Data Flow



- - **Input Devices**→ Sensors, switches, cameras
- - **Processing** → CPU executes instructions
- - **Output Devices** → LEDs, motors, displays, actuators
- - **Communication** → USB, Wi-Fi, Bluetooth

1.10. Building Blocks

An embedded system is built from several key **hardware components**, each playing a specific role:

1.10.1. Central Processing Unit (CPU)

- The **brain** of the system.
- Can be a **microcontroller, microprocessor, DSP, or SoC**.
- Executes instructions and controls other hardware components.

1.10. Building Blocks

1.10.2. Memory

- **ROM / Flash** → Stores the **firmware** (permanent program).
- **RAM** → Provides temporary storage for active tasks and variables.

1.10.3. Input Devices

- Allow the system to **receive data** from the outside world.
- **Examples**: sensors (temperature, pressure, motion), switches, keyboards, cameras.



1.10. Building Blocks

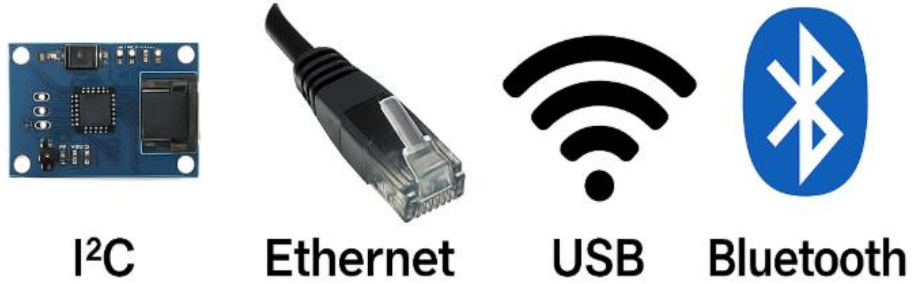
1.10.4. Output Devices

- Provide results or feedback from the system.
- **Example**: LEDs, displays, speakers, motors, actuators.



1.10.5. Communication Interfaces

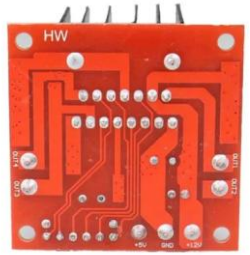
- Enable **data transfer** between the embedded system and other devices.
- **Example**: I²C, CAN, Ethernet, USB, Wi-Fi, Bluetooth.



1.10. Building Blocks

1.10.6. Application-Specific Circuitry

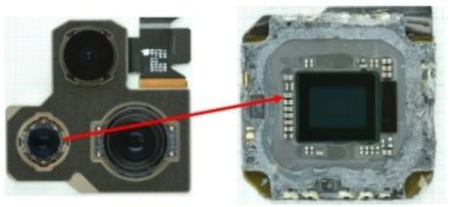
- Custom hardware designed for the specific purpose of the device.
- **Example**: Motor driver circuits in a washing machine, camera sensor interface in a smartphone.



Circuits



Camera



Sensor and interface

1.11. Embedded Processors

1.11.1. Microcontrollers (MCU)

- A **single chip** containing CPU, memory, and input/output (I/O).
- Low-cost and power-efficient.
- Common in small devices (washing machines, toys, IoT sensors).
- **Examples**: STM32, AVR, PIC.



STM32

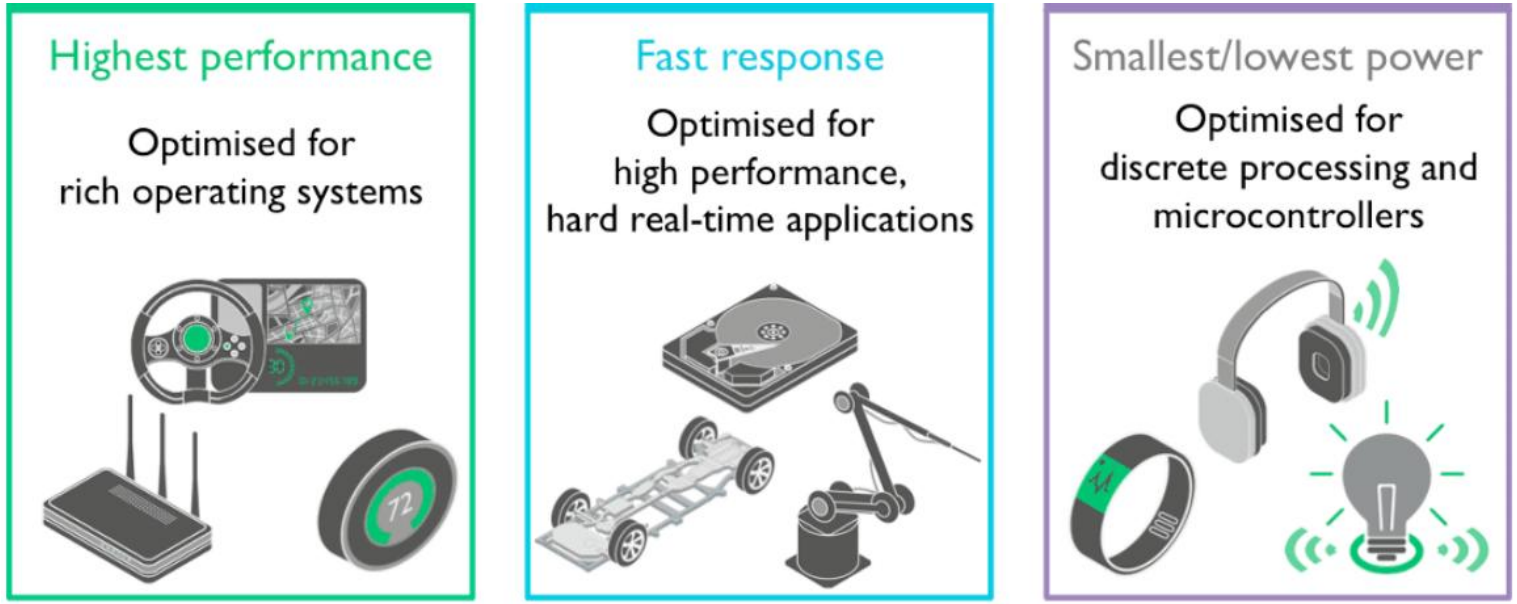


AVR

1.11. Embedded Processors

1.11.2. Microprocessors (MPU)

- More powerful than MCUs, usually need **external memory** and peripherals.
- Used in complex systems like smartphones, tablets, or networking devices.
- Examples: **ARM Cortex-A series, Intel Atom**.



1.11. Embedded Processors

1.11.3. Digital Signal Processors (DSPs)

- Specialized for **mathematical operations on signals**.
- Used in audio, video, image, and radar systems.
- **Example**: Qualcomm DSP in mobile phones.



DSP

1.11. Embedded Processors

1.11.4. System-on-Chip (SoC)

- Combines **CPU + GPU + memory + communication units** on a single chip.
- Very common in **smartphones, smart TVs, and modern embedded devices**.
- Example: **Apple A-series chips, Snapdragon SoC**.



Apple A-series



Snapdragon SoC

1.12. Embedded Operating Systems

1.12.1. Bare-Metal Systems

- No operating system → the software directly controls the hardware.
- Very fast and efficient, but harder to program for complex tasks.

1.12.2. RTOS (Real-Time Operating System)

- Provides **task scheduling** and guarantees responses within strict time limits.
- Used in **time-critical applications** (automotive, robotics, medical devices).

1.12.3. Embedded Linux / Android

- Provides **multitasking, file systems, networking, and user interfaces**.
- Suitable for **complex, feature-rich devices** (smartphones, routers, smart TVs).

1.13. Development Tools & Methods

1.13.1. Cross-Compilation Toolchains

- Special compilers that generate code for the **target hardware** (not your PC).

1.13.2. Integrated Development Environments (IDEs)

- Provide a complete environment for **coding, compiling, and debugging**.

1.13.3. Debugging Tools

- Allow developers to **test and trace code on hardware**.

1.14. Programming Languages

1.14.1. Assembly Language

- A **low-level programming language** that is closest to machine code but still human-readable.
- **Platform-Specific** → Each type of processor (e.g., ARM, AVR, x86) has its own assembly instructions.
- **Direct Hardware Control** → Provides fine-grained control over CPU registers, memory, and hardware.
- **No Compilation Needed** → Instructions are directly translated into machine code through an **assembler**.

1.14. Programming Languages

1.14.2. C Language

- **Modular Programming:** Programs can be broken into multiple source files and can linked together.
- **Low-Level Access:** Allows direct interaction with hardware.
- **Easy to Learn & Use:** Simpler syntax compared to assembly.
- **Portability:** Can run on different hardware platforms with minimal changes.
- **Efficiency:** Produces fast and memory-efficient code when compiled.

1.15. Classification

1.15.1. Small Scale Embedded System

1.15.2 Medium Scale Embedded System

1.15.3 Sophisticated Embedded System

1.15.1. Small-Scale Embedded Systems

Processor Type:

- Typically use a **single 8-bit or 16-bit microcontroller**.
- Example: Intel 8051, AVR, or PIC microcontrollers.

Complexity:

- Hardware and software are relatively **simple** compared to large-scale systems.

Power Consumption:

- Many small-scale systems are **battery-operated**.
- Must be designed to **minimize power dissipation**, especially when running continuously.

Programming Language:

- Most applications are developed using **C language** because of its efficiency and portability.
- Assembly language may be used for **low-level control** in some cases.

1.15.1. Small-Scale Embedded Systems

Examples of Small-Scale Embedded Systems:



Digital watches



Microwave ovens



Simple calculators



Toys with microcontrollers

1.15.2. Medium-Scale Embedded Systems

Processor Type:

- Use **single or multiple 16-bit / 32-bit microcontrollers**,
- Or **Digital Signal Processors (DSPs)** for high-speed data processing.

Complexity:

- Hardware and software are **more complex** than in small-scale systems.
- Suitable for devices requiring **multi-tasking** and faster performance.

Programming Tools:

- **RTOS (Real-Time Operating System)** → manages tasks and timing.
- **Source Code Engineering Tools** → for structured development.
- **Simulator & Debugger** → for testing before deployment.
- **Integrated Development Environment (IDE)** → e.g., Keil MDK, STM32CubeIDE, IAR.

1.15.2. Medium-Scale Embedded Systems

Examples of Medium-Scale Embedded Systems:



Digital cameras (handiing image capture, storage, processing)



Smartphones (older generations) with single-core processors



Advanced washing machines with multiple programs andd sensors



Automotive infotainment systems

1.15.3 Sophisticated Embedded Systems

Complexity:

- Involve **enormous hardware and software complexity**.
- Designed for highly advanced applications requiring **high processing power and specialized functionality**.

Processor Requirements:

- Often need **scalable processors** or **configurable/custom processors**.
- Use **programmable logic devices (FPGAs, ASICs)** for flexibility and performance.

Programming Challenges:

- High development cost and effort due to **customized hardware-software integration**.

1.15.3 Sophisticated Embedded Systems

Examples of Sophisticated Embedded Systems:



Avionics systems in aircraft (flight control computers. radar)



Missile guidance systems



Supercomputing--based automotive systems (autonomous vehicles)



Satellite and space systems

1.16. Summary

System → A set of components working together to achieve a goal.

Embedded System → A computer with hardware + software built for a specific task.

Characteristics → Dedicated, real-time, resource-limited, reliable, low power.

Applications → Found in consumer, automotive, medical, industrial, IoT, and aerospace.

Advantages → Cost-effective, compact, efficient, reliable, and power-saving.

1.16. Summary

Hardware Blocks → CPU, memory, input/output devices, interfaces, custom circuits.

Operating Systems → Bare-metal (simple), RTOS (real-time), Linux/Android (complex).

Development Tools → Toolchains, IDEs, debuggers, version control, CI/CD.

Languages → Assembly (low-level), C (standard).

